

Principais Requisitos do Desafio	Regra arquitetural geral	Foi avaliado na seleção de teste	Foi avaliado no cenário de teste	Foi teste no cenário de teste	system_name	status	nodes	edges	target_availability	cenário	assumptions_made	semantic_issues	simulation_results	telemetry	Resumo da cobertura deste teste
Cenário de Degradação (what-if com matemática)	Lentidão em dependência síncrona causa estouro de timeout e amplificação de carga via retries.	Seleção 1	1. Cenário: Degradação de Latência (degradation)	Caso 1.1: Degradação com estouro de timeout	order-system	sucesso_com_alertas	orders-api (4 rep), payments (1 rep), mysql-primary (1 rep)	orders-api -> payments (sync, 2 retries, 3000ms timeout)	0.999	degradation: mysql-primary (4000ms)	Assumido 1 réplica (payments, mysql), call_type sync	SPOF (payments, mysql), Amplificação de Carga, Risco de Cascata	Timeout excedido (4050ms), Carga amplificada 3x, mysql-primary 250% saturação. Disp cai.	2158.63ms 1955 tokens	☑ CORRETO. Cálculo preciso do impacto de retries em saturação.
Cenário de Degradação	Sem retries, o impacto se reflete em indisponibilidade por esgotamento, mas não em amplificação de RPS.	Seleção 1	1. Cenário: Degradação de Latência (degradation)	Caso 1.2: Degradação sem retries (Cascata simples)	Gateway-Auth	sucesso_com_alertas	gateway (1 rep), authentication (2 rep)	gateway -> auth (sync, 0 retries, 5000ms timeout)	0.99	degradation: authentication (2500ms)	Assumido 0 retries, backoff nulo para auth	SPOF (gateway)	Nenhum timeout excedido. Saturação auth 40%. Dentro da meta.	2157.8ms 2057 tokens	☑ CORRETO. Absorveu a degradação sem cascata destrutiva.
Cenário de Indisponibilidade	Falha em dependência síncrona sem fallback interrompe fluxo crítico.	Seleção 1	2. Cenário: Componente Indisponível (unavailability)	Caso 2.1: Queda de Banco de Dados	product-catalog	sucesso_com_alertas	catalog-service, redis-cluster, postgres	catalog -> redis (sync), catalog -> postgres (sync)	0.999	unavailability: redis-cluster	1 réplica para todos, sem limites RPS explícitos	SPOFs múltiplos, Riscos de cascata	Falha imediata catalog->redis. Disponibilidade cai a zero.	1482.18ms 1749 tokens	☑ CORRETO. Bloqueio absoluto do fluxo síncrono.
Cenário de Indisponibilidade	Filas atuam como amortecedores (Circuit Breaker natural). Produtor não deve cair se consumidor cair.	Seleção 1	2. Cenário: Componente Indisponível (unavailability)	Caso 2.2: Queda de Fila Assíncrona	Checkout System	sucesso_com_alertas	checkout-api, kafka, invoice-worker	checkout -> kafka (async), kafka -> invoice (async)	0.999	unavailability: kafka	Assumidas chamadas async baseadas no texto	SPOFs múltiplos	Worker inativo. Mensagens retidas. Disponibilidade do fluxo kafka cai a zero.	1982.9ms 1971 tokens	☑ CORRETO. Blast radius contido perfeitamente no consumidor assíncrono.
Cenário de Multiplicador de Carga	Escalabilidade vertical/horizontal deve suportar o pico estipulado, do contrário ocorre saturação.	Seleção 1	3. Cenário: Carga de Entrada Multiplicada	Caso 3.1: Pico de tráfego (Black Friday)	Web System	sucesso_com_alertas	frontend (10 rep), cart-api (15 rep, 30 rps)	frontend -> cart-api (sync)	0.999	load_multiplier: 5x	Call_type sync para frontend -> cart	Risco de Cascata	Pico estimado: 2000 rps. Cart-api satura em 444%.	1642.94ms 1781 tokens	☑ CORRETO. Gargalo matemático identificado e comprovado.
Lidar com entradas fora do escopo	Recusa amigável sem inventar dados	Seleção 1	4. Cenário: Validações Adversariais	Caso 4.1: Fora do escopo suportado	N/A	recusado_fora_de_escopo	N/A	N/A	N/A	unsupported (memory leak)	N/A	N/A	Recusado: Pergunta fora do escopo (latencia/rps)	1627.27ms 1791 tokens	☑ CORRETO. LLM acionou gatilho de unsupported adequadamente.
Validação Golden Test (Matemática Pura)	Gabarito: 4x amplificação, 300% saturação no Postgres	Seleção 2	Cenário 1: Degradação de Latência	Teste 1.1: O Exemplo Canônico	checkout_system	sucesso_com_alertas	checkout, pricing, postgres	checkout -> pricing (sync, 3 retries)	0.999	degradation: postgres (3000ms)	postgres (1 rep, irrestrito)	SPOF (postgres), Amplificação, Risco Cascata	Pricing satura 75%, Postgres 300%, Amplifica 4x (960 rps).	1983.26ms 1894 tokens	☑ CORRETO. Bateu exatamente com o gabarito derivado à mão.
Validação Golden Test	Gabarito: DB capacidade irrestrita absorve pico 2x	Seleção 2	Cenário 3: Multiplicador de Carga	Teste 3.2: Pico Absorvido	Catalog Service	sucesso_com_alertas	catalog-api (10 rep, 50 rps), catalog-db	catalog-api -> catalog-db (sync)	0.999	load_multiplier: 2x	catalog-db (irrestrito: max_rps=null)	SPOF, Risco Cascata	Catalog-api 100%, catalog-db 625%.	1369.75ms 1677 tokens	⚠️ ATENÇÃO. O motor assumiu 80 RPS default no código Python para DB irrestrito, gerando falso positivo de falha. Requer ajuste de premissa no código.
Lidar com Contradições	Gabarito: LLM resolve ambiguidade (Fila = Async, anula timeout HTTP)	Seleção 2	Cenário 4: Validações Adversariais	Teste 4.3: Texto Contraditório	KafkaWorkerSystem	sucesso_com_alertas	worker, kafka	kafka -> worker (sync, timeout 5000ms)	0.999	unavailability: kafka	Assumido sync para consumo kafka, timeout 5s	SPOF	Kafka cai a zero.	2017.83ms 1953 tokens	❌ FALHA ARQUITETURAL NO PARSER. O LLM mapeou um broker Kafka como 'sync' com timeout HTTP, violando regras de design. O prompt precisa corrigir o usuário.